LAPORAN PRAKTIKUM

TEKNOLOGI IoT 2

Protokol Komunikasi Telemetri Industri (MQTT)

DOSEN :

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T. Agus Ambarwari, S.Pd., M.Kom.

Disusun Oleh :

Nama : Adelia Rahmadani (25782030)

Kelas : TRI 3B

PROGRAM STUDI TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2026

Latihan 1 (Analisis Arsitektur & Perbandingan Protokol)

Soal: Pada Modul 3, kita menggunakan protokol HTTP untuk membaca dan mengontrol perangkat. Jelaskan secara mendalam mengapa protokol MQTT jauh lebih unggul daripada HTTP dalam hal:

1. Efisiensi penggunaan kapasitas bandwidth pada paket data seluler.
2. Beban komputasi mikrokontroler (CPU overhead) saat menangani pertukaran data dua arah.
3. Kemampuan pengiriman data secara simultan ke banyak perangkat penerima (1-to-N distribution).

Jawab: Protokol MQTT unggul signifikan dibandingkan HTTP dalam hal efisiensi bandwidth pada jaringan seluler karena menggunakan struktur biner yang amat ringkas dengan ukuran *fixed header* minimal hanya dua byte. Sebaliknya, HTTP yang berbasis teks memiliki *overhead header* sangat berat, yakni sekitar 200 hingga 800 byte per transaksi, akibat kewajiban membawa informasi metode, status, dan berbagai atribut tambahan berulang kali.

Dari segi beban komputasi (*CPU overhead*), HTTP memakai model *Request-Response* yang bersifat *stateless*, memaksa mikrokontroler membuka dan menutup koneksi TCP secara terus-menerus pada setiap pengiriman data sehingga membebani pemrosesan CPU dan SRAM. Di sisi lain, MQTT menjaga koneksi TCP tetap persisten melalui mekanisme *Keep-Alive Ping* yang sangat ringan, dipadu arsitektur *Publish-Subscribe* serta fungsi *callback* asinkron sehingga perangkat hanya memproses data saat ada paket yang masuk tanpa mengganggu alur utama program.

Dalam hal distribusi data satu ke banyak penerima (*1-to-N distribution*), HTTP mengandalkan komunikasi *point-to-point* langsung yang mewajibkan pengirim melakukan *request* terpisah ke tiap aplikasi pemantau. Sementara itu, MQTT memanfaatkan *MQTT Broker* sebagai perute pesan terpusat, di mana satu kali pengaliran data dari node pengirim akan otomatis disebarkan secara instan ke seluruh *subscriber* tanpa memberatkan mikrokontroler.

Latihan 2 (Analisis Hierarki Topik & Karakter Wildcard)

Soal: Sebuah pabrik manufaktur memiliki 3 lini produksi (line1, line2, line3), di mana setiap lini memiliki 4 mesin (m1, m2, m3, m4), dan setiap mesin melaporkan metrik suhu, getaran, dan status. Struktur topiknya adalah `pabrik/<line>/<mesin>/<sensor>`. Tentukan ekspresi topik subskripsi (*subscription topic*) yang tepat menggunakan karakter wildcard (`+` atau `#`) untuk:

i. Memantau parameter suhu dari seluruh mesin di semua lini produksi.

- **Ekspresi Topik:** `pabrik/+/+/suhu`
- **Penjelasan:** Memanfaatkan *single-level wildcard* (`+`) pada segmen lini (*line1, line2, line3*) dan segmen mesin (*m1, m2, m3, m4*), serta mengunci tingkat akhir secara spesifik pada atribut `suhu`.

ii. Memantau seluruh parameter sensor (suhu, getaran, status) khusus untuk mesin m1 di lini line2.

- **Ekspresi Topik:** `pabrik/line2/m1/#`
- **Penjelasan:** Mengunci rute secara spesifik pada `line2` dan mesin `m1`, lalu menerapkan *multi-level wildcard* (`#`) di ujung topik guna menangkap seluruh hierarki parameter sensor di bawahnya (*suhu, getaran, status*).

iii. Memantau seluruh lalu lintas data tanpa terkecuali dari seluruh lini dan seluruh mesin di pabrik tersebut.

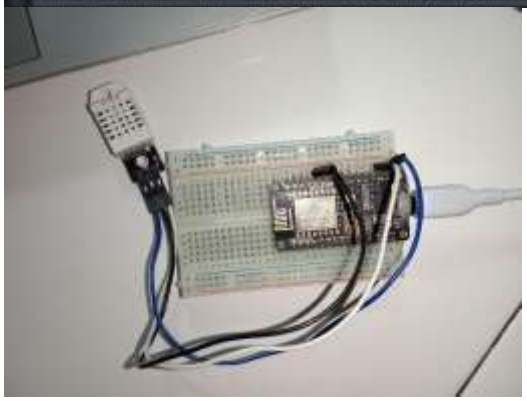
- **Ekspresi Topik:** `pabrik/#`
- **Penjelasan:** Menempatkan *multi-level wildcard* (`#`) langsung setelah direktori utama `pabrik/` untuk menyadap seluruh pesan, cabang hierarki, lini, mesin, hingga jenis sensor secara menyeluruh.

Latihan 3 (Implementasi Serialisasi Data Dinamis)

Soal: Modifikasi blok penyusunan JSON pada Praktikum 1 agar paket telemetri menyertakan dua parameter tambahan berikut:

1. `"uptime_detik"`: Lama waktu aktif sistem dalam satuan detik (dihitung dari rumus `millis() / 1000`).
2. `"ip_lokal"`: Alamat IP lokal mikrokontroler dalam format string (`WiFi.localIP().toString()`).





- **`uptime_detik` (`millis() / 1000`):** Berfungsi mencatat durasi masa aktif sistem sejak mikrokontroler dihidupkan atau mengalami *reset*. Fungsi bawaan `millis()` menghitung waktu dalam satuan milidetik yang terus meningkat secara berkelanjutan, sehingga pembagian angka 1000 diperlukan untuk konversi ke satuan detik. Metrik ini penting pada aplikasi pemantauan perangkat untuk menilai keandalan sistem serta mendeteksi pemadaman listrik atau kegagalan daya mendadak.
- **`ip_lokal` (`WiFi.localIP().toString()`):** Berfungsi melampirkan alamat IP lokal yang dialokasikan oleh router Wi-Fi ke modul ESP8266 dalam wujud teks *string*. Hal ini memudahkan tim teknis dalam melacak alamat identitas jaringan tempat perangkat terhubung secara langsung melalui paket data yang diterima.

Latihan 4 (Analisis Mekanisme Reconnect & Loop Blocking)

Soal: Pada Praktikum 3, di dalam fungsi `loop()` terdapat pemanggilan rutin `client.loop()`.

i. Apa konsekuensi teknis yang terjadi terhadap responsivitas penerimaan perintah callback jika seorang pengembang menyisipkan instruksi `delay(6000)` di dalam fungsi `loop()`?

Jawab: Jika perintah `delay(6000)` disisipkan di dalam fungsi `loop()`, mikrokontroler akan berada dalam kondisi *blocking* (berhenti beraktivitas total) selama 6 detik penuh pada setiap putarannya. Dampak teknis yang timbul meliputi:

- **Kegagalan *Keep-Alive Ping*:** Rutinitas `client.loop()` tidak dapat dipanggil secara konsisten untuk memelihara sesi TCP dengan broker MQTT, menyebabkan broker menganggap perangkat putus akibat batas waktu (*timeout*).
- **Penurunan Responsivitas Drastis:** Perintah kendali yang dikirimkan oleh pengguna (misalnya mengubah status nyala LED) tidak bisa diterima atau dieksekusi secara instan (*asinkron*), melainkan terhambat sampai siklus *delay* usai dan koneksi berhasil tersambung kembali.

ii. Jelaskan bagaimana fungsi `reconnect()` bekerja memulihkan koneksi saat router Wi-Fi atau broker MQTT sempat terputus sesaat!

Jawab: Fungsi `reconnect()` bekerja melalui perulangan bersyarat menggunakan evaluasi perintah `client.connected()`. Saat jalur transmisi terputus:

- Sistem mendeteksi kondisi kegagalan lalu menampilkan kode kesalahan (*return code/rc*) ke Serial Monitor untuk keperluan diagnostik.
- Fungsi ini secara aktif mengupayakan jabat tangan ulang (*handshake*) ke broker menggunakan panggilan `client.connect(clientId)`.
- Sesaat setelah sambungan berhasil dipulihkan, fungsi ini langsung mengeksekusi kembali perintah pendaftaran topik (*subscription*) seperti `client.subscribe()`, sehingga perangkat kembali siap menerima instruksi kontrol tanpa perlu memuat ulang (*restart*) sistem secara manual.